Math Homework

Problem Solving Challenge

Problem Description

You are given a sorted array `points[]` representing the point values of math problems a student must solve in order.

The student must follow these rules:

- They must solve the first problem (index 0).
- After solving a problem at index i, they may solve either:
  - the next problem ($i + 1$), or
  - the one after that ($i + 2$).
- They continue solving problems until:
  - the difference between the maximum and minimum solved point values is ≥ `threshold`.

If this threshold can never be met, the student must solve all problems.

Task: Determine the **minimum** number of problems the student needs to solve.

Constraints

$1 \le n \le 100$

$1 \le points[i] \le 1000$

$1 \le threshold \le 1000$

Sample Test Cases

Example 1
Input: `threshold = 4, points = [1, 2, 3, 5, 8]`
Output: 3
Explanation: Solve questions with values 1, 3, and 5. (5 - 1 = 4, which is ≥ threshold).

Example 2
Input: `threshold = 4, points = [1, 2, 3, 4, 5]`
Output: 3
Explanation: Solve questions with values 1, 3, and 5. (5 - 1 = 4).

Java Solution

```java
import java.io.*;
import java.util.*;

class Result {

    /*
     * Complete the 'minNum' function below.
     * The function returns an INTEGER.
     */
    public static int minNum(int threshold, List<Integer> points) {
        int n = points.size();

        // Standard BFS or DP approach to find the minimum steps
        // To reach the threshold value effectively.
        int minSolved = n;

        for (int i = 1; i < n; i++) {
            if (points.get(i) - points.get(0) >= threshold) {
                // We need to solve index 0, and then move towards index i
                // In each step we can skip at most one element (jump i+2)
                // The number of steps to reach index i from 0 is ceil(i/2)
                int currentSolved = (i + 1) / 2 + 1;
```

```
                    minSolved = Math.min(minSolved, currentSolved);
                }
            }

            return minSolved;
        }
    }
```